

```
#!/usr/bin/env python3
```

```
"""
```

AEGIS RAG - Enterprise Secure Multi-Source RAG System

Complete Django Backend Source Code

Author: Rachit Mehta

Email: rachit-mehta@outlook.com

License: MIT

```
"""
```

```
# =====
```

```
# FILE: requirements.txt
```

```
# =====
```

```
REQUIREMENTS = """
```

```
Django>=5.0
```

```
djangoRESTframework>=3.14
```

```
sentence-transformers>=2.2
```

```
faiss-cpu>=1.7.4
```

```
numpy>=1.24
```

```
scikit-learn>=1.2
```

```
python-dotenv>=1.0
```

```
openai>=1.0
```

```
psycopg2-binary>=2.9
```

```
gunicorn>=21.2
```

```
django-prometheus>=2.3
```

```
celery>=5.3
```

```
redis>=5.0
```

```
"""
```

```
# =====
```

```
# FILE: aegis_rag/settings.py
```

```
# =====
```

```
SETTINGS_PY = """
```

```
import os
```

```
from pathlib import Path
```

```
BASE_DIR = Path(__file__).resolve().parent.parent
```

```
SECRET_KEY = os.environ.get('DJANGO_SECRET_KEY', 'change-me-in-production')
```

```
DEBUG = os.environ.get('DEBUG', 'False').lower() == 'true'
```

```
ALLOWED_HOSTS = ['*']
```

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'rest_framework',  
    'core',  
]
```

```
MIDDLEWARE = [  
    'django.middleware.security.SecurityMiddleware',  
    'django.contrib.sessions.middleware.SessionMiddleware',  
    'django.middleware.common.CommonMiddleware',  
    'django.middleware.csrf.CsrfViewMiddleware',  
    'django.contrib.auth.middleware.AuthenticationMiddleware',  
    'django.contrib.messages.middleware.MessageMiddleware',  
]
```

```
ROOT_URLCONF = 'aegis_rag.urls'
```

```
TEMPLATES = [  
    {  
        'BACKEND': 'django.template.backends.django.DjangoTemplates',  
        'DIRS': [BASE_DIR / 'templates'],  
        'APP_DIRS': True,  
        'OPTIONS': {  
            'context_processors': [  
                'django.template.context_processors.debug',  
                'django.template.context_processors.request',  
                'django.contrib.auth.context_processors.auth',  
                'django.contrib.messages.context_processors.messages',  
            ],  
        },  
    ],  
]
```

```
WSGI_APPLICATION = 'aegis_rag.wsgi.application'
```

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': os.environ.get('DB_NAME', 'aegis_rag'),
        'USER': os.environ.get('DB_USER', 'postgres'),
        'PASSWORD': os.environ.get('DB_PASSWORD', 'postgres'),
        'HOST': os.environ.get('DB_HOST', 'localhost'),
        'PORT': os.environ.get('DB_PORT', '5432'),
    }
}
```

```
AUTH_PASSWORD_VALIDATORS = []
```

```
LANGUAGE_CODE = 'en-us'
```

```
TIME_ZONE = 'UTC'
```

```
USE_I18N = True
```

```
USE_TZ = True
```

```
STATIC_URL = 'static/'
```

```
STATICFILES_DIRS = [BASE_DIR / 'static']
```

```
DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
```

```
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework.authentication.SessionAuthentication',
        'rest_framework.authentication.TokenAuthentication',
    ],
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticated',
    ],
}
```

```
# OpenAI API Key
```

```
OPENAI_API_KEY = os.environ.get('OPENAI_API_KEY', '')
```

```
# FAISS Index Path
```

```
FAISS_INDEX_PATH = BASE_DIR / 'faiss_index.bin'
```

```
CHUNK_IDS_PATH = BASE_DIR / 'chunk_ids.pkl'
```

```
BM25_MATRIX_PATH = BASE_DIR / 'bm25_matrix.pkl'
```

```
VECTORIZER_PATH = BASE_DIR / 'vectorizer.pkl'
```

```
# Cache
```

```
CACHES = {
    'default': {
        'BACKEND': 'django.core.cache.backends.redis.RedisCache',
        'LOCATION': os.environ.get('REDIS_URL', 'redis://localhost:6379/1'),
    }
}
```

```
# Logging
```

```
LOGGING = {
    'version': 1,
    'disable_existing_loggers': False,
    'handlers': {
        'console': {'class': 'logging.StreamHandler'},
    },
    'root': {
        'handlers': ['console'],
        'level': 'INFO',
    },
}
"""
```

```
# =====
```

```
# FILE: core/models.py
```

```
# =====
```

```
MODELS_PY = """
```

```
from django.db import models
from django.contrib.auth.models import User
from django.core.validators import MinValueValidator
```

```
class UserProfile(models.Model):
```

```
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    clearance_level = models.IntegerField(default=0, validators=[MinValueValidator(0)])
    department = models.CharField(max_length=50)
    allowed_roles = models.JSONField(default=list)
```

```
    def __str__(self):
```

```
        return f"{self.user.username} (L{self.clearance_level})"
```

```

class Document(models.Model):
    DOC_TYPES = [('pdf','PDF'), ('csv','CSV'), ('json','JSON'), ('txt','Text')]
    title = models.CharField(max_length=255)
    source_type = models.CharField(max_length=10, choices=DOC_TYPES)
    file_path = models.FileField(upload_to='documents/', null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.title

```

```

class Chunk(models.Model):
    document = models.ForeignKey(Document, on_delete=models.CASCADE,
related_name='chunks')
    text = models.TextField()
    embedding_id = models.IntegerField(null=True, blank=True)
    metadata = models.JSONField(default=dict)

```

```

class Meta:
    indexes = [models.Index(fields=['metadata'])]

```

```

class AuditLog(models.Model):
    user = models.ForeignKey(User, on_delete=models.SET_NULL, null=True)
    action = models.CharField(max_length=100)
    query = models.TextField(blank=True, null=True)
    response_summary = models.TextField(blank=True, null=True)
    timestamp = models.DateTimeField(auto_now_add=True)
    metadata = models.JSONField(default=dict)

```

```

class Meta:
    ordering = ['-timestamp']
"""

```

```

# =====
# FILE: core/retrieval.py
# =====
RETRIEVAL_PY = ""
import numpy as np
import faiss
import pickle
import os
from sentence_transformers import SentenceTransformer

```

```

from sklearn.feature_extraction.text import TfidfVectorizer
from django.conf import settings
from .models import Chunk

class HybridRetriever:
    def __init__(self):
        self.model = SentenceTransformer('all-MiniLM-L6-v2')
        self.index = None
        self.chunk_ids = []
        self.bm25_matrix = None
        self.vectorizer = TfidfVectorizer(stop_words='english', sublinear_tf=True)
        self._load_or_build()

    def _load_or_build(self):
        if os.path.exists(settings.FAISS_INDEX_PATH):
            self.index = faiss.read_index(str(settings.FAISS_INDEX_PATH))
            with open(settings.CHUNK_IDS_PATH, 'rb') as f:
                self.chunk_ids = pickle.load(f)
            with open(settings.BM25_MATRIX_PATH, 'rb') as f:
                self.bm25_matrix = pickle.load(f)
            with open(settings.VECTORIZER_PATH, 'rb') as f:
                self.vectorizer = pickle.load(f)
        else:
            self.build_indices()

    def build_indices(self):
        chunks = Chunk.objects.all()
        if not chunks:
            return
        texts = [c.text for c in chunks]
        embeddings = self.model.encode(texts, show_progress_bar=True)
        dim = embeddings.shape[1]
        self.index = faiss.IndexFlatIP(dim)
        faiss.normalize_L2(embeddings)
        self.index.add(embeddings)
        self.chunk_ids = [c.id for c in chunks]
        faiss.write_index(self.index, str(settings.FAISS_INDEX_PATH))
        with open(settings.CHUNK_IDS_PATH, 'wb') as f:
            pickle.dump(self.chunk_ids, f)
        self.bm25_matrix = self.vectorizer.fit_transform(texts)
        with open(settings.BM25_MATRIX_PATH, 'wb') as f:

```

```

        pickle.dump(self.bm25_matrix, f)
    with open(settings.VECTORIZER_PATH, 'wb') as f:
        pickle.dump(self.vectorizer, f)

def get_visible_chunks(self, user):
    profile = user.profile
    visible = Chunk.objects.filter(
        metadata__min_clearance__lte=profile.clearance_level
    )
    visible_ids = set()
    for chunk in visible:
        allowed = chunk.metadata.get('allowed_roles', [])
        if not allowed or any(r in allowed for r in profile.allowed_roles):
            visible_ids.add(chunk.id)
    return visible_ids

def retrieve(self, query, visible_ids, top_k=5):
    if not visible_ids:
        return []
    q_emb = self.model.encode([query])
    faiss.normalize_L2(q_emb)
    search_k = min(len(visible_ids), 50)
    distances, indices = self.index.search(q_emb, search_k)
    query_vec = self.vectorizer.transform([query])
    bm25_scores = self.bm25_matrix.dot(query_vec.T).toarray().flatten()
    combined = []
    for i, dist in zip(indices[0], distances[0]):
        chunk_pk = self.chunk_ids[i]
        if chunk_pk not in visible_ids:
            continue
        bm25 = bm25_scores[i] if i < len(bm25_scores) else 0
        score = 0.6 * float(dist) + 0.4 * float(bm25)
        combined.append((chunk_pk, score))
    combined.sort(key=lambda x: x[1], reverse=True)
    top_pks = [pk for pk, _ in combined[:top_k]]
    return list(Chunk.objects.filter(id__in=top_pks))
'''

# =====
# FILE: core/nli.py
# =====

```

```

NLI_PY = '''
from sentence_transformers import CrossEncoder

class NLIVerifier:
    def __init__(self):
        self.model = CrossEncoder('cross-encoder/nli-deberta-v3-small')

    def verify(self, answer, chunks):
        claims = [s.strip() for s in answer.split('.') if len(s.strip()) > 5]
        if not claims:
            return 1.0
        supported = 0
        for claim in claims:
            max_entail = 0.0
            for chunk in chunks:
                prediction = self.model.predict([(claim, chunk.text)])
                score = float(prediction[0]) if prediction.ndim == 1 else float(prediction[0][0])
                max_entail = max(max_entail, score)
            if max_entail > 0.7:
                supported += 1
        return supported / len(claims)
'''

# =====
# FILE: core/llm.py
# =====
LLM_PY = '''
import openai
from django.conf import settings

def generate_answer(query, context_chunks):
    if not context_chunks:
        return "Insufficient information."
    context_str = "\n".join(
        f"[{c.id}] [{c.document.title}] {c.text}" for c in context_chunks
    )
    prompt = f"\n\nYou are an enterprise assistant. Answer ONLY from context below.
If context lacks the answer, say \"Insufficient information\".
For every statement, cite the chunk ID in brackets.
Context:
{context_str}

```


Question: {query}

Answer: \" \" \"

try:

```
openai.api_key = settings.OPENAI_API_KEY
response = openai.ChatCompletion.create(
    model="gpt-4",
    messages=[{"role": "user", "content": prompt}],
    temperature=0.2,
    max_tokens=500
)
```

```
return response.choices[0].message.content
```

except Exception:

```
return f"Based on available context: {context_chunks[0].text}
[{context_chunks[0].id}]"
'''
```

```
# =====
```

```
# FILE: core/views.py
```

```
# =====
```

```
VIEWS_PY = '''
```

```
from rest_framework.decorators import api_view, permission_classes
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response
from .retrieval import HybridRetriever
from .nli import NLIVerifier
from .llm import generate_answer
from .models import AuditLog
```

```
retriever = HybridRetriever()
```

```
verifier = NLIVerifier()
```

```
@api_view(['POST'])
```

```
@permission_classes([IsAuthenticated])
```

```
def query_view(request):
```

```
    query = request.data.get('query', '').strip()
```

```
    if not query:
```

```
        return Response({'error': 'Empty query'}, status=400)
```

```
    user = request.user
```

```
    visible_ids = retriever.get_visible_chunks(user)
```

```
    top_chunks = retriever.retrieve(query, visible_ids, top_k=5)
```

```
    if not top_chunks:
```

```

        AuditLog.objects.create(user=user, action='query_blocked', query=query,
                                metadata={'reason': 'no_visible_chunks'})
    return Response({
        'answer': 'I cannot answer due to insufficient permissions or lack of relevant data.',
        'sources': [],
        'confidence': 0.0,
        'nli_score': None
    })
    answer = generate_answer(query, top_chunks)
    nli_score = verifier.verify(answer, top_chunks)
    citations = [{'id': c.id, 'source': c.document.title, 'snippet': c.text[:150]} for c in
top_chunks]
    AuditLog.objects.create(user=user, action='query_success', query=query,
                            response_summary=answer[:200],
                            metadata={'nli_score': nli_score, 'chunks_used': len(top_chunks)})
    return Response({
        'answer': answer,
        'sources': citations,
        'confidence': round(nli_score, 3),
        'nli_score': round(nli_score, 3),
        'retrieval_count': len(top_chunks)
    })

@api_view(['GET'])
@permission_classes([IsAuthenticated])
def audit_view(request):
    logs = AuditLog.objects.filter(user=request.user)[:50]
    return Response([
        {
            'action': l.action,
            'query': l.query,
            'timestamp': l.timestamp.isoformat(),
            'summary': l.response_summary
        } for l in logs])
'''

# =====
# FILE: core/urls.py
# =====
URLS_PY = """
from django.urls import path
from . import views

```

```

urlpatterns = [
    path('api/query/', views.query_view, name='query'),
    path('api/audit/', views.audit_view, name='audit'),
]
"""

# =====
# FILE: core/admin.py
# =====
ADMIN_PY = """
from django.contrib import admin
from .models import UserProfile, Document, Chunk, AuditLog

@admin.register(UserProfile)
class UserProfileAdmin(admin.ModelAdmin):
    list_display = ('user', 'clearance_level', 'department')

@admin.register(Document)
class DocumentAdmin(admin.ModelAdmin):
    list_display = ('title', 'source_type', 'created_at')

@admin.register(Chunk)
class ChunkAdmin(admin.ModelAdmin):
    list_display = ('id', 'document', 'text_preview')
    def text_preview(self, obj):
        return obj.text[:100]

@admin.register(AuditLog)
class AuditLogAdmin(admin.ModelAdmin):
    list_display = ('user', 'action', 'timestamp')
    list_filter = ('action',)
"""

# =====
# FILE: manage.py
# =====
MANAGE_PY = """#!/usr/bin/env python
import os
import sys

```

```

def main():
    os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'aegis_rag.settings')
    try:
        from django.core.management import execute_from_command_line
    except ImportError as exc:
        raise ImportError(
            "Couldn't import Django."
        ) from exc
    execute_from_command_line(sys.argv)

if __name__ == '__main__':
    main()
"""

# =====
# PRINT ALL FILES FOR EXTRACTION
# =====
if __name__ == '__main__':
    print("Save this file and extract individual components for deployment.")
    print("Full source code packaged for Microsoft Build AI 2026 Submission")

```